

# Desenvolvimento de um Motor Orquestrador Nativo em C para Processamento Concorrente e Pipelines de Dados em Sistemas Unix

Gabryel M. Assis<sup>1</sup>

<sup>1</sup>Departamento de Ciência da Computação – Universidade Federal de Roraima (UFRR)  
– Boa Vista – RR – Brazil

[gabryelmartins777@gmail.com](mailto:gabryelmartins777@gmail.com)

**Abstract.** *This article details the architecture, data structures, and native POSIX system calls employed in building a process orchestration engine (edge-engine). Developed strictly in C, the system was designed for Edge Computing devices, where resource management and the mitigation of memory failures are critical. The documentation covers the separation between lexical analysis and orchestration, the lifecycle and isolation of concurrent processes, the mitigation of zombie processes via asynchronous signal handling, the in-memory geometry for inter-process communication (pipes), and the safe handling of character encoding. Dynamic testing confirmed the system's integrity under stress, with no recorded memory leaks.*

**Resumo.** *Este artigo detalha a arquitetura, as estruturas de dados e as chamadas de sistema nativas POSIX empregadas na construção de um motor orquestrador de processos (edge-engine). Desenvolvido estritamente em linguagem C, o sistema foi desenhado para dispositivos de Computação de Borda (Edge Computing), onde o gerenciamento de recursos e a mitigação de falhas de memória são críticos. A documentação abrange a separação entre análise léxica e orquestração, o ciclo de vida e isolamento de processos concorrentes, a mitigação de processos fantasmas via tratamento assíncrono de sinais, a geometria em memória para comunicação interprocessos (pipes), e o tratamento seguro de codificação de caracteres. Testes dinâmicos confirmaram a integridade do sistema sob estresse, sem o registro de vazamentos de memória.*

## 1. Introdução

Com a consolidação de arquiteturas voltadas para a Computação de Borda, o processamento analítico próximo à fonte de dados exige infraestruturas de software de latência ultrabaixa. Diferente de frameworks tradicionais executados em máquinas virtuais de alto nível, os motores operacionais nativos comunicam-se diretamente com o kernel do sistema. O objetivo deste projeto foi desenvolver um interpretador capaz de gerenciar pipelines de dados complexos através da criação isolada de processos e redirecionamento de descritores diretamente na memória, evitando o uso de operações de I/O desnecessárias no armazenamento físico.

## 2. Análise Arquitetural do Terminal

Para garantir eficiência e modularidade, a arquitetura do orquestrador foi dividida em dois subsistemas independentes: o Parser (Analisador Léxico e Sintático) e o Executor

(Orquestrador de Processos). O Parser atua como a interface de entrada no laço REPL (Read-Eval-Print Loop). Sua responsabilidade é receber a cadeia bruta de caracteres (linha de entrada) e fragmentá-la matematicamente em tokens. Utilizando a rotina `strtok`, o analisador substitui delimitadores (como espaços e tabulações) por bytes nulos (`\0`), gerando um vetor dinâmico de ponteiros sem a necessidade de duplicar excessivamente as strings na memória. O Executor, por sua vez, recebe essa árvore sintática simplificada e toma decisões de roteamento. Ele é isolado da leitura do teclado e responsabiliza-se unicamente por verificar a presença de diretivas concorrentes (como `fork` para assincronismo ou `pipe` — para pipes), acionando as chamadas de sistema adequadas para manipular o kernel.

### 3. Estudo de Casos e Testes Representativos

O motor orquestrador foi submetido a um estresse controlado em ambiente Linux nativo via WSL. A seguir, apresentam-se os casos de teste e as respectivas saídas do interpretador, comprovando as restrições de baixo nível exigidas para a infraestrutura de borda.

**A. Processamento Síncrono e Expansão POSIX** Ao executar um comando contendo metacaracteres, a API nativa `glob()` interceptou o curinga antes da injeção no ciclo do processo e expandiu a instrução internamente para localizar o arquivo fonte. O terminal principal permaneceu bloqueado corretamente aguardando a finalização da tarefa em *foreground*.

Plaintext

```
gabryel@Gabryel: /ProjetoFinalSO./edge-engine$ edge-engine > ls*.cmain.c
```

**B. Concorrência de Dados na Memória (Pipelines)** O teste de IPC (Comunicação Interprocessos) demonstrou o funcionamento da geometria de *pipes* acoplada à chamada `dup2()`. O interpretador roteou a saída do programa `cat` diretamente para a entrada do programa `grep`. A comunicação ocorreu estritamente em memória, filtrando os dados sem gerar arquivos temporários no disco físico.

Plaintext

```
edge-engine$ cat Makefile | grep gcc CC = gcc (CC)(CFLAGS) -o (TARGET)main.c
```

**C. Limpeza Assíncrona e Sinais (Mitigação Anti-Zombie)** Para atestar a resiliência contra processos fantasmas (*zombies*), injetou-se uma diretiva de suspensão com o operador lógico de *background* (`&`). O orquestrador retornou o controle da interface reativa instantaneamente. O processo executou nos bastidores e, ao finalizar, sua memória foi varrida silenciosamente através do tratador de sinais assíncrono acoplado ao `SIGCHLD`.

Plaintext

```
edge-engine$ sleep 5 [Processo assíncrono iniciado com PID 4821] edge-engine$ echo "O terminal esta livre!"O terminal esta livre!
```

**D. Encerramento e Auditoria de Memória** Para certificar o cumprimento da exigência de eliminação de vazamentos, o laço de repetição foi quebrado através do comando *built-in* interno. A execução sob a ferramenta de auditoria dinâmica comprovou a total higienização dos ponteiros dinâmicos.

Plaintext

```
edge-engine; exit Encerrando orquestrador... ==4822== HEAP SUMMARY:  
==4822== in use at exit: 0 bytes in 0 blocks ==4822== total heap usage: 15 allocs, 15  
frees, 2,048 bytes allocated ==4822== All heap blocks were freed – no leaks are possible
```

#### 4. Mecanismo de Ciclo de Vida de Processos

O sistema gerencia o ciclo de vida dos comandos solicitados utilizando a mecânica clássica de clonagem e substituição de contexto adotada por sistemas Unix. O fluxo exato opera em duas etapas inseparáveis: A chamada de sistema `fork()` é invocada pelo orquestrador para criar uma cópia exata do processo atual. O momento em que a memória do processo pai é isolada ocorre imediatamente após o retorno positivo do `fork()`, onde o sistema operacional aloca uma nova estrutura de tarefa (`taskstruct`) para o processo filho, mapeando tabelas de páginas exclusivas através do mecanismo de Copy-on-Write (CoW). No espaço de endereçamento do processo filho (onde o retorno do `fork` é 0), invoca-se a chamada `execvp()`. Esta rotina descarta o segmento de texto (código) e o segmento de dados copiados do orquestrador, substituindo-os pelo binário do novo programa. O processo pai permanece ileso, mantendo seu laço de repetição ativo e aguardando a finalização da tarefa.

#### 5. Engenharia de Concorrência e Sinais

A execução de tarefas em background introduz o risco de degradação de recursos por processos fantasmas (zombies). Quando um processo filho é encerrado assincronamente, seu bloco de controle permanece na tabela do kernel até que o processo pai colete seu código de saída (`exit status`). Para mitigar essa proliferação sob altas cargas de execução, implementou-se o tratamento assíncrono do sinal `SIGCHLD`. A implementação matemática e lógica deste tratamento reside em uma estrutura de repetição que utiliza a chamada `waitpid` combinada com a flag `WNOHANG`: `while(waitpid(-1, NULL, WNOHANG) > 0);` A lógica desse laço fundamenta-se no comportamento não-enfileirado dos sinais POSIX. Se múltiplos processos em concorrência terminarem no mesmo milissegundo, o sistema operacional pode aglutinar a notificação, enviando apenas um `SIGCHLD` ao pai. O laço `while` força o orquestrador a verificar ativamente a tabela de processos e realizar o reaproveitamento de memória (reaping) de todos os filhos terminados simultaneamente, prevenindo categoricamente a falha de processos zombies.

#### 6. Geometria de Pipes e Redirecionamento

A comunicação interprocessos (IPC) para encadeamento de fluxos de dados foi construída mapeando-se descritores de arquivos ativos através de matrizes de *pipes* lineares. A chamada de sistema `pipe(fd)` aloca um canal unidirecional de dados estritamente no *buffer* do *kernel*, devolvendo dois descritores: `fd[0]` (ponta de leitura) e `fd[1]` (ponta de escrita). A técnica de duplicação assíncrona foi consolidada utilizando a rotina `dup2()`. Em um cenário de múltiplos comandos, o processo produtor invoca `dup2(fd[1], STDOUT_FILENO)`, forçando sua saída padrão a injetar dados na memória compartilhada. Simultaneamente, o processo consumidor executa `dup2(fd[0], STDIN_FILENO)`, ingerindo os dados gerados sem qualquer operação de disco. A integridade desta arquitetura depende do encerramento das pontas não utilizadas no espaço isolado de cada processo e, crucialmente, no encerramento de ambas as

cópias do *pipe* no processo pai para evitar a condição letal de interbloqueio cego (*deadlock*).

## 7. Expansão de Metacaracteres (WILDCARDS)

O orquestrador delega o reconhecimento e a expansão de metacaracteres (como `*.txt` ou `log_?`) à API nativa POSIX `glob()` antes da injeção no ciclo de vida do processo. A implementação mapeia iterativamente cada argumento inserido pelo usuário. Ao detectar padrões curingas, a função `glob` é invocada com o parâmetro estrutural `GLOB_NOCHECK`. O retorno da função gera um vetor expansível contendo os caminhos absolutos dos arquivos resolvidos. Para manter a estabilidade da memória dinâmica, a alocação do vetor resultante obedece a um algoritmo de escalonamento usando `realloc()`, duplicando fisicamente cada *string* descoberta através de `strdup()`. Uma rotina emparelhada de purgação de memória (`free()`) foi acoplada ao final do ciclo do orquestrador para garantir o expurgo completo dos caminhos alocados dinamicamente após a execução do comando.

## 8. Codificação de Caracteres e Internacionalização

O processamento de fluxos massivos exige cautela quanto à codificação. O tratamento de cadeias de caracteres em sistemas Unix modernos deve considerar as diferenças entre o padrão ASCII (que ocupa 1 byte por caractere, de tamanho previsível) e o UTF-8 (que possui largura variável de 1 a 4 bytes por caractere). O impacto de se utilizar *buffers* de tamanho estático (ex: `char buffer[1024]`) com a função `fgets()` é o risco agudo de *buffer overflow* ou truncamento silencioso de dados, especialmente se o usuário injetar comandos longos compostos por caracteres UTF-8. Para mitigar corrupções na tabela de *strings* e vazamentos, o interpretador implementou a função dinâmica POSIX `getline()`. Esta rotina readapta internamente o ponteiro de entrada através de readequações baseadas no comprimento real da cadeia de bytes, blindando o ambiente interativo de falhas de limite e preservando a internacionalização da entrada do sistema.

## 9. Portfólio de Comandos Suportados

No paradigma de Sistemas Operacionais, os comandos *built-in* não podem ser delegados a processos filhos via `fork()`. Se o orquestrador submetesse a instrução "mudar de diretório" a um filho, apenas o contexto clonado seria alterado, sem impacto real para o interpretador. O atual escopo do orquestrador suporta a seguinte diretiva nativa processada diretamente em seu núcleo:

`exit|`: Aciona o encerramento gracioso do ambiente reativo. Quando detectado pela etapa do *Parser*, ele impede imediatamente a criação de subprocessos, invoca todas as funções de descarte de memória alocada (`free`) do *loop* principal e retorna um código de saída padrão (0) ao *kernel* original, garantindo zero pendências sistêmicas.

## 10. Referências

[1] SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Sistemas Operacionais: Conceitos e Aplicações. Rio de Janeiro: LTC, 9ª Edição, 2015. [2] TANENBAUM, A. S.; BOS, H. Sistemas Operacionais Modernos. São Paulo: Pearson, 4ª Edição, 2016. [3] KERISK, M. The Linux Programming Interface. No Starch Press, 2010. [4] STEVENS, W.

R.; RAGO, S. A. Advanced Programming in the UNIX Environment. Addison-Wesley, 3rd Edition, 2013.

Agradecimentos: O autor expressa seu agradecimento à ferramenta Gemini 3.1 Pro da Google pelo suporte na revisão linguística e aprimoramento estilístico deste manuscrito. Todo o conteúdo conceitual, interpretação de dados e redação final foram rigorosamente supervisionados e validados pelos autores humanos, que assumem total responsabilidade pelo texto.”Versão em Inglês (Padrão para Periódicos Internacionais / SBC):

Acknowledgements: The authors acknowledge the use of Gemini Pro 3.1 developed by Google for linguistic revision and style editing of this manuscript. All conceptual content, data interpretation, and final writing were strictly supervised and validated by the human authors, who bear full responsibility for the published work.

Materiais e Métodos [Ambiente Experimental e Ferramentas]: O desenvolvimento e a otimização dos scripts de orquestração de processos concorrentes e encadeamento de pipelines de dados em linguagem ANSI C foram realizados com o suporte assistido do modelo de linguagem de grande porte Google Gemini. A ferramenta foi empregada especificamente para a estruturação do laço interativo (REPL), implementação da comunicação interprocessos via geometria de pipes, expansão de metacaracteres (API POSIX glob) e estruturação do tratamento assíncrono de sinais para mitigação de processos zumbis. Ressalta-se que toda a lógica algorítmica, parametrização e validação lógica do código gerado foram executadas e auditadas manualmente pelo autor

Materials and Methods: The development and optimization of the source code for concurrent process orchestration and in-memory data pipelines in ANSI C were supported by the large language model Google Gemini. This tool was specifically applied for structuring the interactive loop (REPL), implementing inter-process communication via pipe geometry, expanding wildcards (POSIX glob API), and setting up asynchronous signal handling for zombie process mitigation. The underlying algorithmic logic, parameters, and logical validation of the generated code were completely reviewed and verified by the author